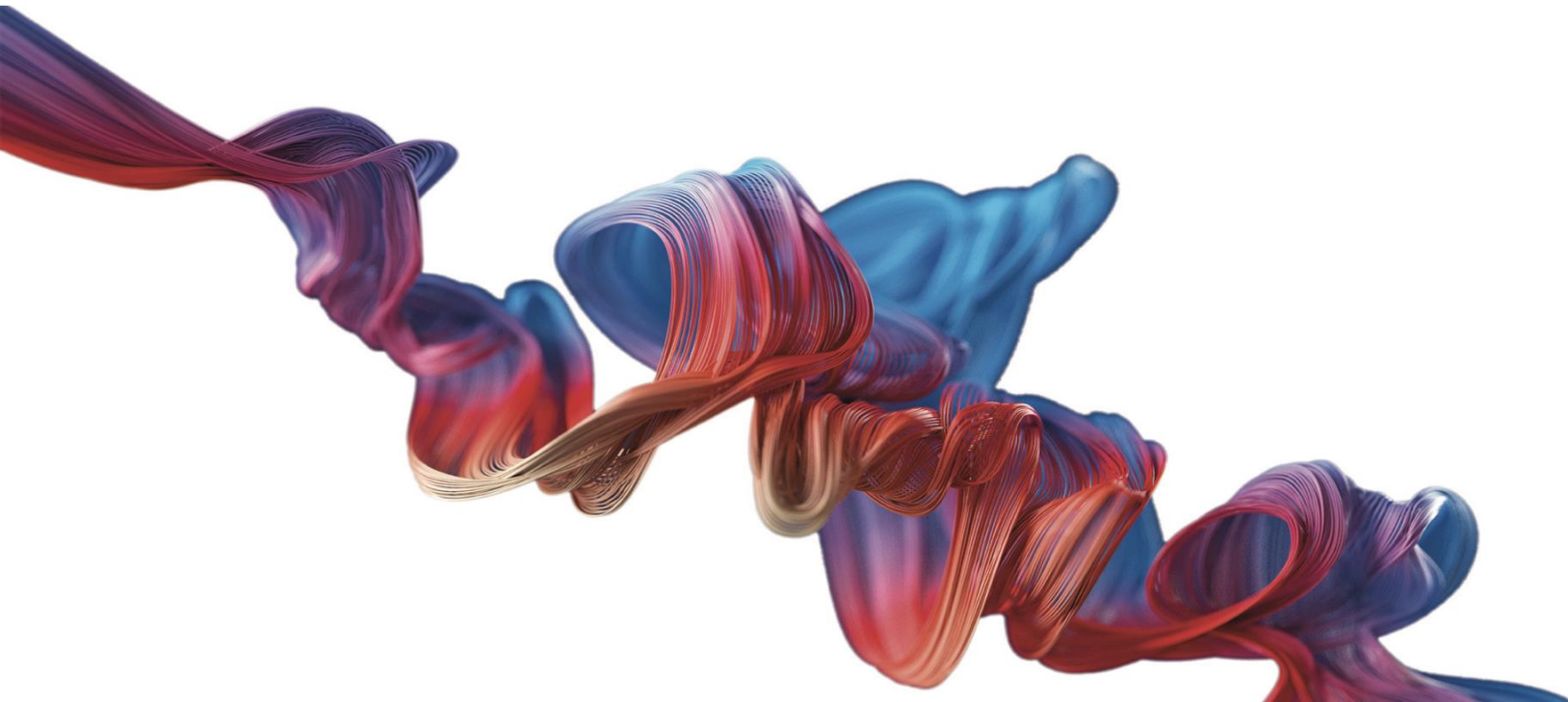


FROM SOCIAL SCIENCE TO DATA SCIENCE

By Bernie Hogan

Chapter 13. Introducing network analysis



Learning goals

- Appreciate that networks have distinct types (e.g., directed or undirected, uniplex or multiplex, unweighted or weighted);
- Be able to create and modify a network object in Python using `networkx`;
- Understand how to make a network plot;
- Understand how to create a subplot of only the largest component and why;
- Be able to generate a network from thread replies on Stack Exchange.

Connections that signal social structure

Network analysis considers relationships between entities such as people, accounts, groups, countries, books, or webpages. Effectively, anywhere that we can define a set of entities (which we commonly call 'nodes') and a set of relationships that link these nodes (which we commonly call 'edges'), we can produce a network.

Some examples of networks available with online data:

- Friendships / following relationships on Facebook, Twitter, or LinkedIn
- Commenting behaviour on a forum where there is a link from a reply to an earlier comment.
- A co-similarity network indicating who purchases similar books
- A co-similarity network indicating where people participate in similar forums, such as those who post in the same subreddits.

Networks can be created from many data sources. Some relations, like comments, might be more distinct than others, like 'best friend'. Distinct relations often come with a lot of noise whereas less distinct relations often come with a lot of variance.

Theories of social networks

Social network theories tend to cluster into a number of domains, each with their own interests and expectations. This chapter does not focus much on network theories as on practice, but some primers might still be useful:

- **Social capital:** We often have interest in who can access what resource in a network. Social capital is a way of characterizing whether the network itself facilitates cohesion as well as characterizing the relative advantage of any node in the network.
- **Similarity and identity:** Networks can be used to identify individuals of similar interests or focus. This can be through people who have similar connections (i.e. many friends in common) or who share similar interests (i.e. read the same books or are cited in the same papers).
- **Relational cartography:** We sometimes simply want to get a sense of who is in a domain or group. Networks are a powerful way to characterize a set of relations and provide these maps.

Network analysis and Python (and R)

Python has capable packages for network analysis. However, more sophisticated routines in social sciences are generally found in R. That said, Python is still very suitable for data cleaning, exploratory work, and many common network routines. Also Python has great routines for one task seen later: community detection.

Two popular networks packages in Python are `igraph` and `networkx`. Since `networkx` is bundled with Anaconda Jupyter it is worth using, certainly to start. Begin with:

```
import networkx as nx
```

This loads the library which includes objects for different kinds of networks.

From this library you can visualize networks, do rudimentary network calculations, and add complex sets of attributes. The development of `networkx` has been focused on making graph querying simple and Pythonic and leaving more complex routines to other packages such as the `community` package.

But first we must determine what sort of network to create...

The edges that make the graph

A relation between two nodes can be:

- a **state** (Ali and Barb are (or not) friends; Ali and Cam (did or did not) attend class together)
- a **flow** (Ali talks to Barb; Ali shares gossip with Cam, Ali sends Dot some cupcakes).

In `networkx` we can create graphs that are undirected (representing states) or directed (representing flows).

```
g1= nx.Graph()  
g2= nx.DiGraph()
```

Between any two nodes could be one relationship type or multiple. So in `networkx` we can also create a `MultiGraph` (for multiple states) and `MultiDiGraph` (for multiple flows).

```
g3 = nx.MultiGraph()  
g4 = nx.MultiDiGraph()
```

Creating network graphs I:

Adding Nodes

Adding nodes in networkx can be done one by one, as a batch, or from a file. We start with a variable that will be the 'network object', `g`. To add nodes one by one:

```
g.add_node(<node_name>); g.add_node(<node_name>)
```

These will not add any connections or attributes, but the graph will now have two nodes. You can check this with:

```
g.info()
```

There are many ways to read an existing network. The simplest is to use a common format like graphml. For example: `g = nx.read_graphml(<path_to_file>)`

You can also add multiple nodes at the same time by adding a collection:

```
g.add_nodes_from(<list_of_nodeids>)
```

This adds nodes to the object, it does not return a new object with the added nodes.

Creating network graphs II:

Adding edges

Similar to nodes, we can add a single edge, collections of edges, or read edges as a part of a file. Each edge is just two nodes as a tuple or collection.

Single edge: `g.add_edge(("Ali", "Barb"))`

Multiple edges: `g.add_edges_from([("Ali", "Barb"), ("Ali", "Cam"), ("Ali", "Dot")])`

From a file: `nx.read_graphml(<path>)`, `nx.read_csv(<path>)`, etc...

Notice the double parenthesis? That's a collection `("Ali", "Barb")` within a function `g.add_edge()`.

- If the graph is directed then it will be from Ali to Barb. You would need `("Barb", "Ali")` to get the reciprocal edge.
- If the graph is multiplex then it will throw an error – it needs to know which edge type. In that case, `g.add_edge(("Ali", "Barb"), 1)` will specify an edge of type 1. You can use additional numbers for additional edge types and use attributes to label the types, such as "friend", "coworker".

If an edge contains an unrecognized node, it will add it rather than throwing an error.

Adding and using attributes

Nodes, edges, and even networks can have attributes. In `networkx` these are dictionaries of key:value pairs.

```
g.graph["label"] = "a graph label"
g.nodes[0]["label"] = "a node label"
g.edges[(1,2)]["label"] = "an edge label"
```

The attributes are stored in `networkx` as a dictionary with the object as a key. So if you want the attributes for node "David", it would simply be `g.nodes["Ali"]`.

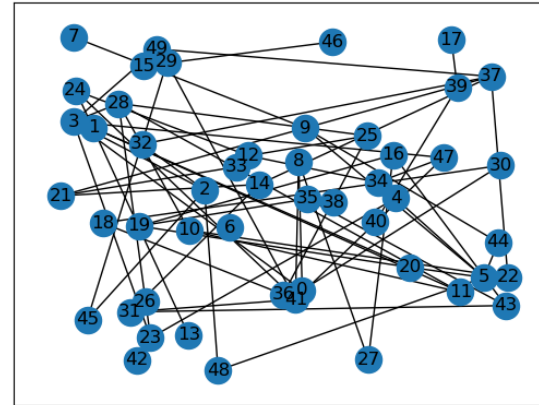
Then to add a new attribute, you treat the node as if it were a dictionary variable. So `g.nodes["Ali"]['weight'] = 50` will now mean that the node has a 'weight' attribute. If it already had a weight attribute, the value is now replaced with 50.

Plotting a graph I: Selecting a layout

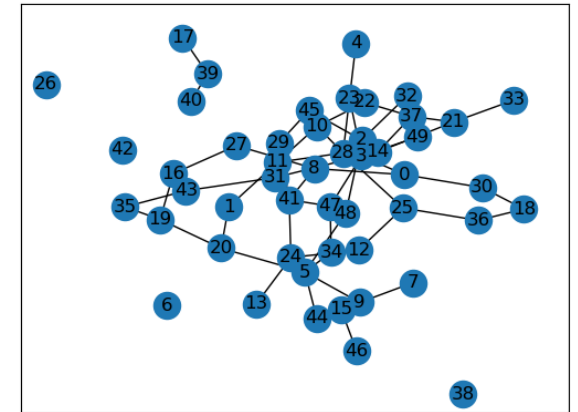
Layouts can make a graph. They have different goals depending on the algorithm.

The classic algorithm by Fruchterman and Reingold shows a model where connected edges get drawn together and disconnected get repelled. This can show clustering well, but not core-periphery structures as they get very dense.

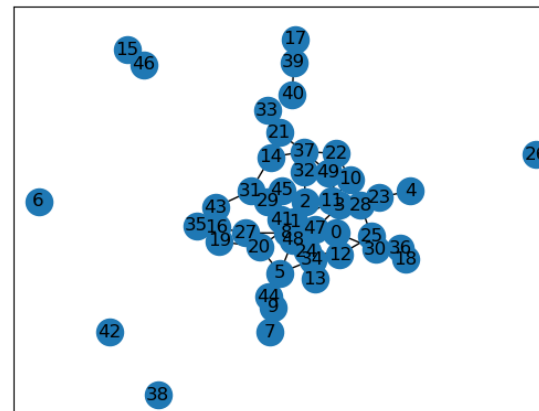
```
pos = nx.spring_layout(g,  
                      iterations=<num>)  
  
nx.draw_networkx(g, pos=pos)  
plt.show() # or  
plt.savefig(<path_to_image>)
```



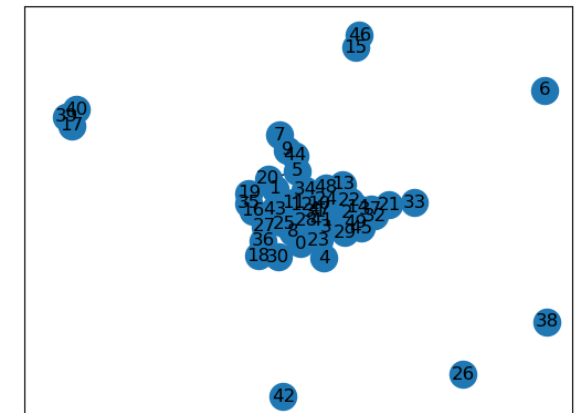
0 iterations (random)



20 iterations



40 iterations



60 iterations

A note on generators

Often networkx will return a “generator” rather than a collection. This is a sort of ‘just-in-time’ collection.

For example, all nodes that are connected to each other are a ‘component’. To get all the components, you can call

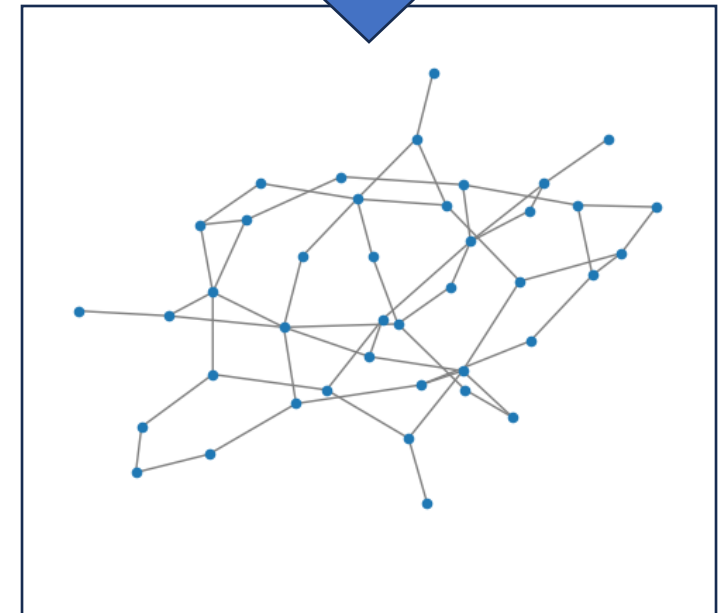
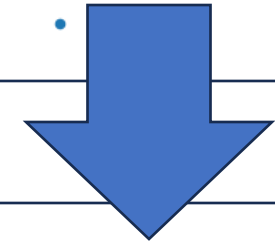
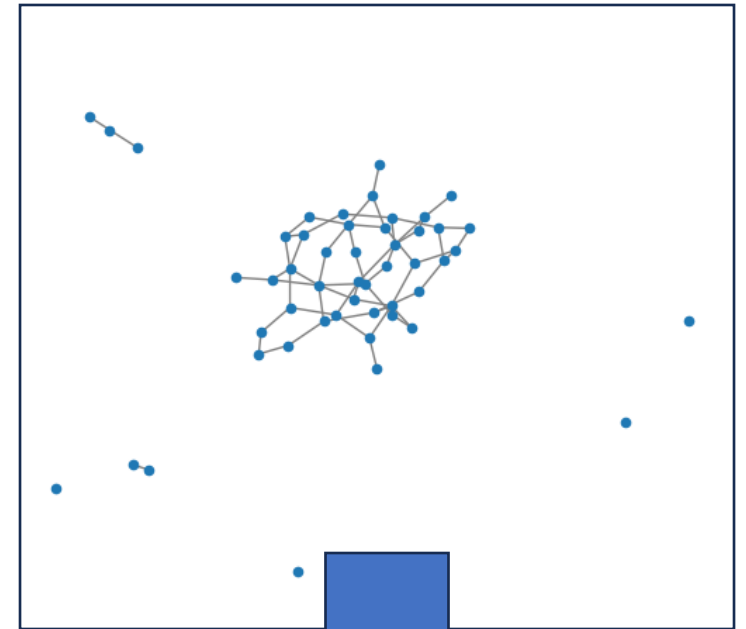
```
comps = nx.connected_components(g)
```

You cannot query a generator like `comps[0]` to find first component, but you can still loop through a generator or make it a list by casting, such as:

```
comps = list(comps)
```

This is really handy to find the largest connected component in a graph. The book shows the code for generating a random network and then getting the largest connected component with

```
max(sorted(nx.connected_components(g),  
          key=len, reverse=True))
```



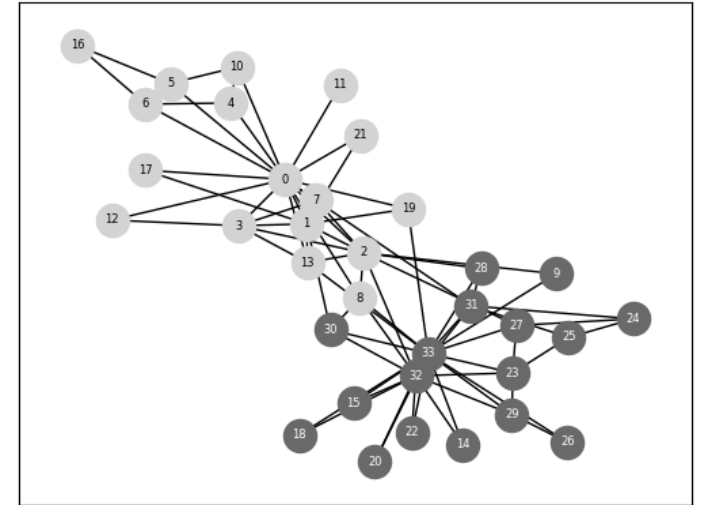
Communities and goodness of fit

Components are clear to see as separate groups.

However, we can also create groups within connected components, called 'communities'. Different algorithms give slightly different solutions though the main patterns tend to be common.

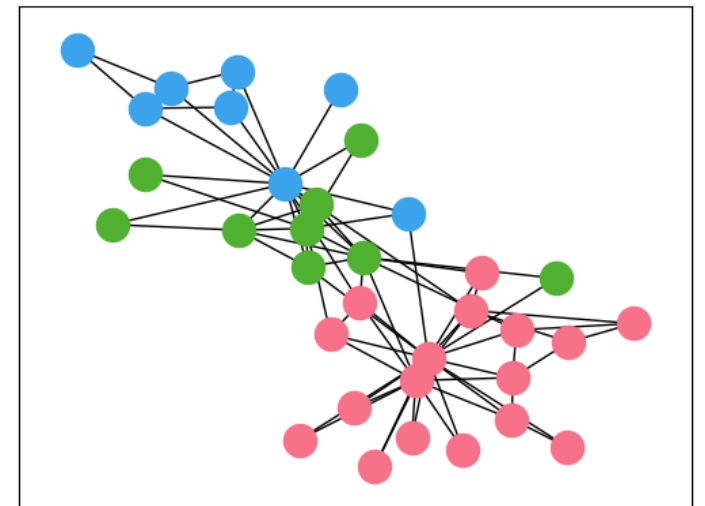
Often the classification is considered useful or interesting when you know what makes the groups. In the classic "Zachary's Karate Club", the two groups were members of the same dojo which then subsequently split into two dojos.

The most common goodness of fit for **community detection** is the score **Modularity** (or Q). $Q > 0.3$ generally means the groups are notable. Modularity scores on networks don't have meaningful p-values.



Above: Labelled data of the two subsequent clubs.

Below: Communities detected that match the labels closely (blue & green to light nodes, pink to dark nodes).



Collecting data:

Network boundaries and missing data

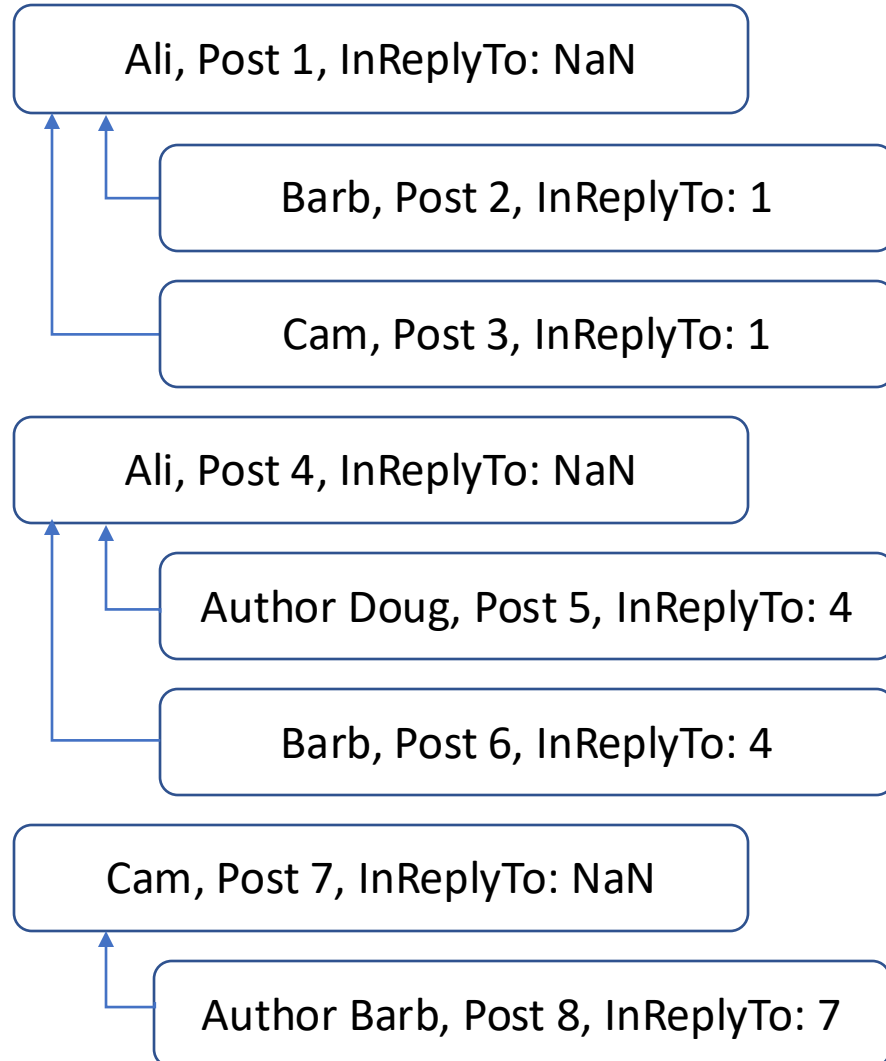
Everything is related somehow. Yet, we use boundaries since we expect some kinds of relationships to be consistent and meaningful when considered together.

Examples of network boundaries:

- All Wikipedia articles; [Whole network]
- All Wikipedia articles about countries in English; [nominal whole network]
- All Wikipedia articles that link to the page for Canada; [egocentric network]
- All people who tweeted about Canada in the last 7 days; [partial network]

A node doesn't have to be a person, but we should be able to define consistent links between nodes. Outside of the boundary of our network are often still nodes. We cannot collect every webpage or every tweet. The boundary can be nominal (such as all countries or all members of a political party) or rational/practical.

Using Stack Exchange I: A whole network using reply data

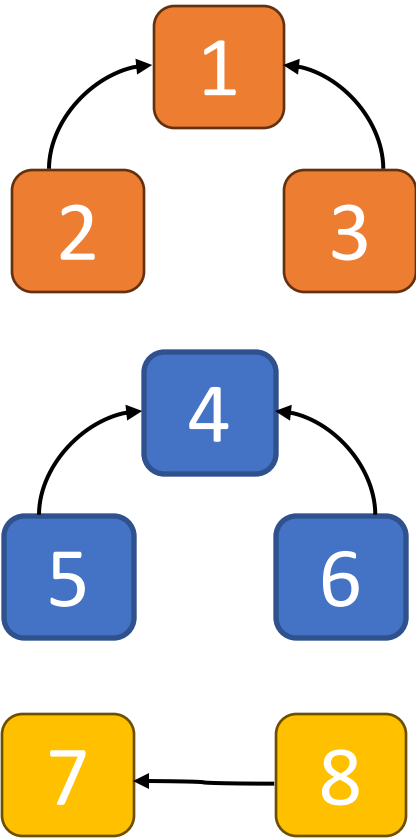


postID	authorID	inreplytoID	inreplytoAuthorID
1	A	NaN	NaN
2	B	1	A
3	C	1	A
4	A	NaN	NaN
5	D	4	A
6	B	4	A
7	C	NaN	NaN
8	B	7	C

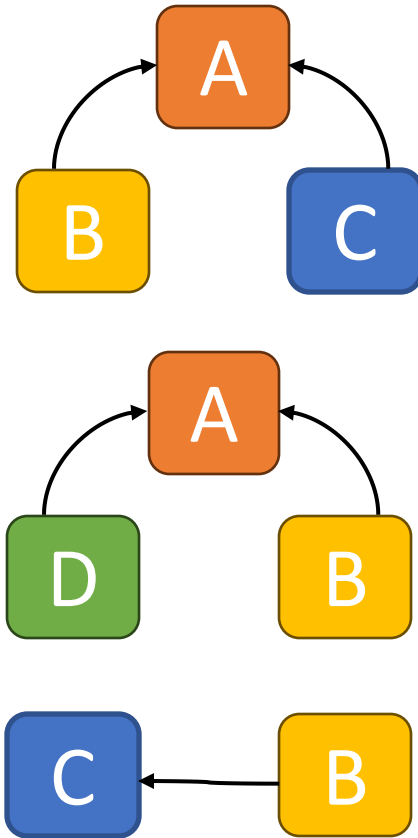
Above: A table listing several ways we can identify a link among posts. We can link post IDs to get threads, but also authorIDs to get a reply graph.

Using Stack Exchange II:

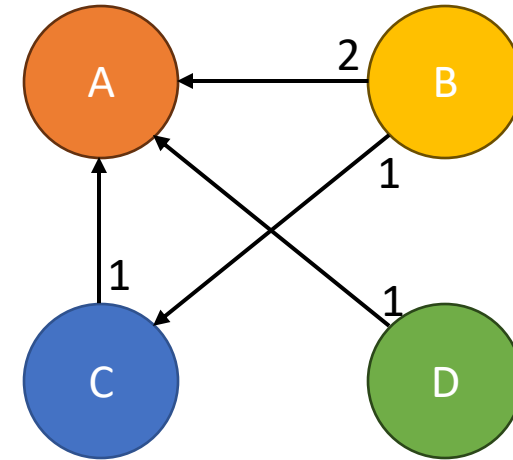
The post network versus the author network



Individual posts as distinct network components



Individual authors identified in the post network



from	to	weight
B	A	1
C	A	1
D	A	1
B	C	1

Authors as nodes in a weighted directed network

Using Stack Exchange: Creating a "similarity" graph

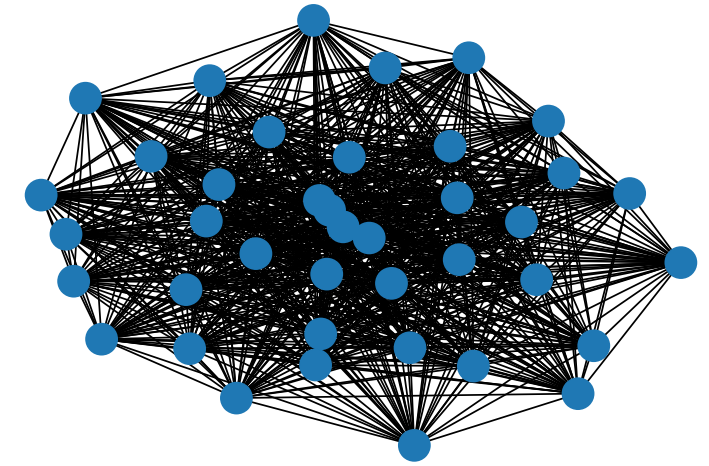
Similarity graphs are common and powerful tools. They are common since you do not need between node links, but can do with links from nodes to a second kind of node.

For example:

- Authors with links to academic citations: who is co-cited a lot?
- Actors to events: Who attends the same classes? The same concerts?

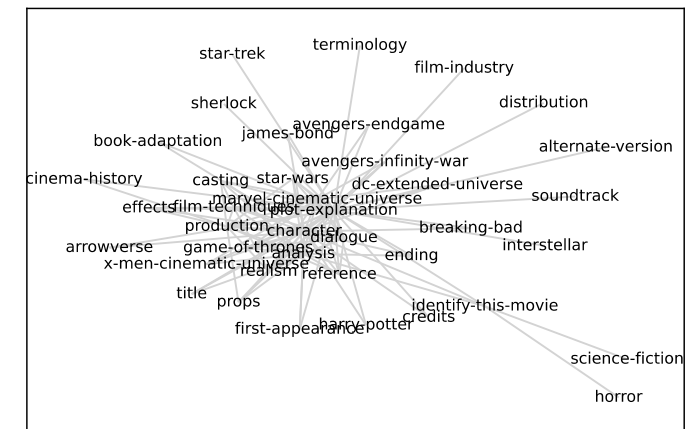
In Stack Exchange we can use the tags. Get a list of tags used by person, then have a link from each person to their tags. `edges = [ ('Author1', 'director'), ...]` Then **project** the graph into author-space or tag-space.

- Weights will be times that two users used the same tags, or the times two tags were used by the same person. If the network is very dense, trim out the tags of weight 1 or 2, since that might not be very informative compared to two people using the same 10 or 20 tags.



Above: A bipartite network of top 40 tags without trimming.

Below: A bipartite network trimmed to 50 or more co-mentions.



Next steps in Network Analysis

These examples merely construct network objects, but do not go in depth into explaining or understanding these. For this there are many specialist books and approaches in networks.

The goal here is to get you thinking about how to wrangle data from one form (tabular) into another form (network) by considering how to link nodes based on relations.

Some key questions now are:

- How did the network come to look this way?
- Are there latent groups and do these change over time? Do they represent interests or conflicts?
- Do people who are better connected behave differently / write differently than those on the periphery?

Summary

Networks are what we take discrete *relations* and structure them in order to understand patterns. These patterns tell us things at the *micro* level (such as reciprocity between people or commenters) and at the *macro* level (such as how communities can be detected between subgroups of a network).

Networks are meant to characterize both *states* and *flows* between nodes. States include friendships (present or absent) whereas flows include information (who tells whom)?

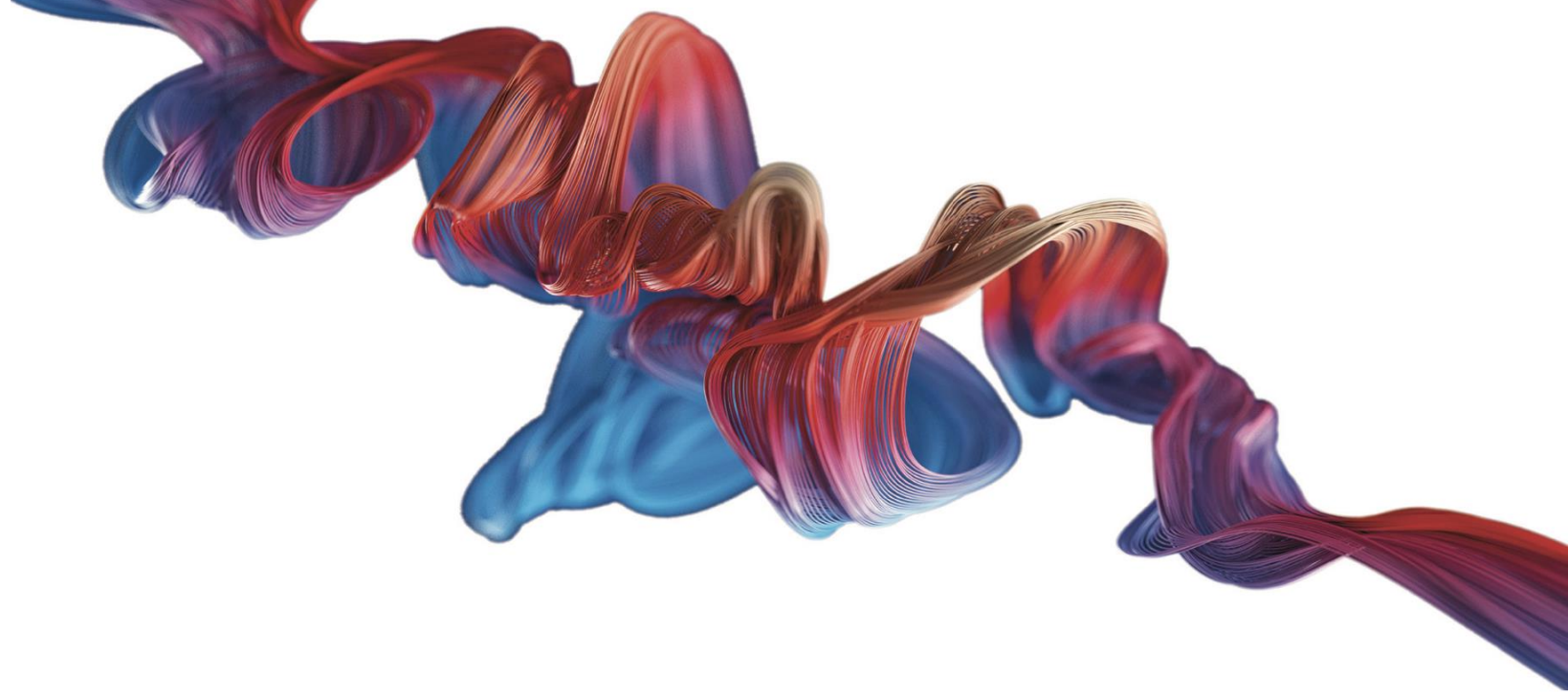
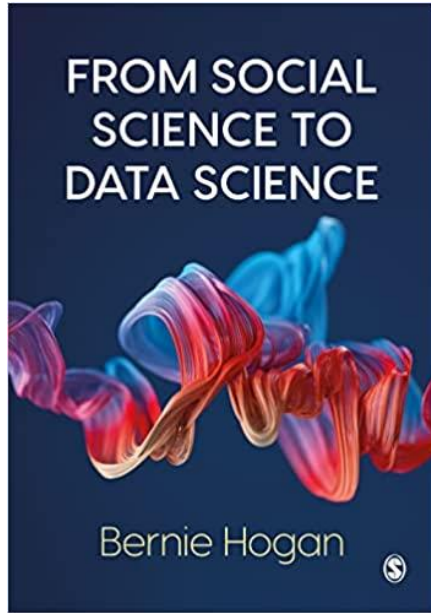
We construct networks usually by wrangling data from a list of relations or from a matrix. We can create network objects, such as a `Graph()` or `DiGraph()` in `networkx`.

Nodes and edges in networks can have *attributes*. These can be meaningful ways to understand the network. This can include attributes from data collection (like a person's Karate club) or from inference (like which community one could be assigned to).

From here there is much to learn about how to ask questions of a network or understand its properties and even model the network statistically.

Questions and extensions

- Explore the generators in `networkx`. They provide a variety of differently shaped networks. Some of the parameters can make a huge difference to the shape of the network.
- Build a network of data from comments on Reddit. Chapter 7 showed how to collect a Reddit page with JSON or with `praw`. Now parse those threads as a network and compare them to other threads. What kinds of topics lead to wide, shallow threads? What kind of topics lead to deep reciprocal conversations?
- Investigate tools for collecting network data from participants. Programs such as Network Canvas (<https://networkcanvas.com/>) simplify the act of interviewing people and getting a rich personal network, but the data that comes out still needs to be wrangled. Complete the sample protocol from Network Canvas (which comes with the program) and then export the data. What do you see? How can you ask questions of that data from multiple people?



Thank you

Up next: Chapter 14. Geographic Information Systems